Problem Solving & Scenario Based Questions

---

1. How do you go about troubleshooting a complex bug? (Walk through your process).

I reproduce the bug reliably and capture exact steps, inputs, environment and timestamps. I collect immediate evidence — logs, stack traces, metrics, error rates — and check recent deploys/config changes, then reduce scope by isolating the smallest possible repro.

I form 2–3 plausible root-cause hypotheses and design quick tests to validate or eliminate each. I instrument selectively (targeted logs, metrics, breakpoints) so I can observe behavior without broad, risky changes.

I implement a tested fix and add unit/integration tests to prevent regression. I roll the change out safely (feature flag or canary) and monitor closely for regressions before full release.

---

2. Explain how you would debug an unfamiliar codebase.

I'd first pair with the author if possible to get quick context, then read the README/architecture docs and run the app locally to observe behaviour.

I'd identify entry points (tests, main modules, relevant routes), use code search to follow the data flow end-to-end, and check git history/PRs for recent changes.

Next I'd add small, non-invasive logs or use the debugger and write tiny unit tests to exercise assumptions.

Finally, I'd take concise notes and update docs so future onboarding is faster.

---

3. What strategies do you use to debug a complex algorithm?.

For debugging a complex algorithm I create minimal reproducible inputs and compare them to small, known-correct examples, then add assertions and invariants at key steps so the code self-checks.

I trace intermediate values with targeted logging or a debugger to see where state diverges, verify edge cases, data types and numeric stability, and break the algorithm into smaller functions to unit-test each piece.

If it's a performance issue I profile hotspots and consider more efficient algorithms or data structures.

---

4. What steps do you take when identifying and solving a coding error?.

When identifying and solving a coding error I reproduce the issue and gather full context — exact inputs, environment, stack traces and logs — then localise the failing component by following the execution path or running module tests.

I add targeted logs/asserts to confirm assumptions, write a failing unit test, implement the fix and run the full test suite plus integration tests.

Finally I submit the change for review, deploy via a safe rollout (feature flag/canary), monitor production and write a short post-incident note.

---

5. How would you evaluate a new feature request to decide whether to implement it?.

I'd first clarify the problem the feature solves and which users and metrics it targets, then check alignment with product goals and the roadmap while surfacing dependencies and risks.

Next I'd consider alternatives or a smaller MVP and whether an experiment can validate our core assumptions. I'd estimate expected impact versus implementation and ongoing maintenance effort, and use prioritisation frameworks like RICE or Eisenhower to compare options.

Finally I'd define measurable acceptance criteria and monitoring, get stakeholder buy-in, and schedule it as prioritised work with a clear rollout strategy (feature flags / staged release).

---

6. How do you decide if a solution is the best approach to a problem? (Weighing pros/cons).

I'd start by listing clear evaluation criteria — correctness, performance, reliability, security, maintainability, time-to-ship and cost — then score each candidate solution against those criteria to expose trade-offs.

I favour simple, well-understood approaches when benefits are comparable, and I explicitly factor in long-term operational costs like tech-debt, monitoring and on-call load. If uncertainty is high I build a small prototype or spike to gather data, and I consult stakeholders and teammates to align goals.

Finally I make the decision transparent, document the reasoning and keep a fallback plan ready.

---

7. Explain how you would optimise an inefficient algorithm. (Profiling, improving complexity).

I'd start by measuring: add benchmarks and profile to confirm the actual slow parts.

Then I'd analyse the algorithmic complexity to spot hotspots like nested loops, repeated computations or unnecessary recursion. I'd replace inefficient data structures with appropriate ones (hash maps, heaps, tries), apply memoisation, caching or precomputation where applicable, and reduce work by removing unnecessary copies or extra iterations—processing elements lazily/streaming where possible. If it's CPU-bound and safe, I'd consider parallelisation or asynchronous processing.

Finally I'd re-run benchmarks and add regression tests/performance benchmarks to ensure the improvements hold.

---

8. Describe your process for identifying performance bottlenecks. (Profiling tools, metrics).

I start by defining clear performance goals such as p50, p95 and p99 latency, throughput, and resource limits.

I then collect logs, system metrics and traces to see where latency concentrates, using OpenTelemetry to send traces to Jaeger and metrics to Prometheus, which I visualise in Grafana.

I use profilers to identify CPU hotspots and memory leaks, and I check database queries with EXPLAIN, network latencies using browser devtools, and third-party service times. I also correlate performance with deployments or traffic changes to spot regressions.

Finally, I prioritise bottlenecks by impact versus cost, apply targeted optimisations, and re-measure to ensure improvements hold.

---

9. What techniques do you use to test and validate your code? (Unit tests, integration tests).

I follow the test pyramid: many fast unit tests, focused integration tests, and a few end-to-end tests. I use mocks and stubs for external dependencies and contract tests for APIs, and add edge case and unexpected input tests when appropriate. I automate tests in CI along with linting, static analysis, and type checks.

I also rely on code review and pair programming for quality and knowledge sharing, and maintain regression tests for reported bugs while adding monitoring and health checks in production.

---

10. What would you do if a production issue occurred during off-hours? / How would you approach a system outage or critical failure?

I acknowledge the incident promptly and follow the runbook or escalation policy. I perform a preliminary assessment to determine severity and quickly implement the safest mitigation, such as a rollback, feature flag, or failover. I keep concise, regular updates to stakeholders and the incident channel, prioritising service restoration before deep diagnosis.

After stabilisation, I perform a blameless postmortem, implement fixes, and update runbooks and alerts to prevent similar issues in the future.

---

11. If you discovered a security vulnerability in your app, how would you respond?

If I discovered a security vulnerability in an app, I would first contain the issue by disabling the vulnerable surface or applying a temporary mitigation without destroying evidence. I would notify the security team and leadership, following responsible disclosure and compliance procedures.

Next, I would perform a preliminary assessment to determine severity, create an action plan for a safe fix, and test it thoroughly in staging. I would then patch the vulnerability, deploy safely, and monitor for any exploitation attempts.

Finally, I would communicate to affected stakeholders per policy and update threat models, tests, and monitoring to prevent similar issues.

---

12. How do you stay calm and focused when debugging under pressure?

When debugging under pressure, I follow a structured checklist — reproduce the issue, isolate the cause, form hypotheses, and test — which helps reduce random attempts and keeps me focused.

I prioritise restoring service with minimal risk through quick mitigation before digging into the root cause. I use pairing to gain a fresh perspective and share the cognitive load, and take short breaks if I get stuck to avoid tunnel vision.

Throughout the process, I communicate clearly and frequently so stakeholders' expectations are managed.

---

13. How do you approach optimising a slow-loading web page?

I start by measuring real user and lab metrics such as First Contentful Paint, Largest Contentful Paint, Time to Interactive, and total byte weight to identify bottlenecks.

I then optimise the critical rendering path by deferring or asynchronously loading non-critical JavaScript, inlining critical CSS, and minimising render-blocking resources. I optimise assets through responsive images, modern formats, and compression, and use caching and a CDN for static assets

alongside server-side improvements like compression and caching. I also lazy-load below-the-fold content and reduce third-party scripts to improve perceived speed.

Finally, I monitor performance to validate improvements across devices and geographies.

---

14. Describe a time you analysed data before making a decision. (e.g. A/B test results).

Project references: 2PD

During development of the administration portal, we needed to decide the best layout for the clinical charting interface, which displays patient sensor data and treatment progress.

I defined the primary metric upfront as time-to-read key metrics for clinicians and secondary metrics as number of errors or misinterpretations. We implemented two layouts (A: detailed table-focused view, B: visual chart-focused view) and ran a small usability test with clinical staff, randomising which version they saw.

I collected metrics on task completion time, accuracy, and subjective satisfaction, and segmented results by device type (desktop vs tablet). After analysing the data, we found that layout B reduced average task time by 22%, reduced errors by 15%, and had higher satisfaction scores.

Based on these results, we chose layout B for rollout, documented the rationale, and monitored adoption post-launch to confirm continued efficiency and accuracy.

---

15. Tell me about a project where you had to think creatively to solve a problem.

Project references: Bare Dating

I was tasked with implementing a tutorial feature that walked users through a story with characters, art, and arrows pointing to interface elements. The challenge was ensuring the arrows and artwork lined up correctly across different screen sizes and devices, given tight timelines and constraints of legacy styling systems.

I proposed two creative approaches: first, using complex styling calculations based on screen dimensions to dynamically position elements; second, implementing an optimisation function to find free space and calculate the shortest path for arrows between elements. I quickly validated both approaches on multiple devices, tracking alignment accuracy and performance.

Ultimately, the optimisation approach was chosen, providing consistent alignment across screens, improving the tutorial’s usability, and reducing styling maintenance. This taught me how to combine creative problem-solving with quick prototyping and rigorous testing to deliver a scalable solution under constraints.